

Documento de Especificação do Projeto — NeuroAnalytics

1. Identificação do Projeto

Nome do projeto: NeuroAnalytics

Cliente: Swift

Tipo de solução: Aplicação web interativa para análise de avaliações de lojas

Área principal: Data Analytics, Business Intelligence e Inteligência Artificial aplicada a NLP

Data de entrega prevista: 08/06/2026

Período da base analisada: Janeiro de 2025 a Janeiro de 2026

2. Visão Geral

O **NeuroAnalytics** será uma aplicação web desenvolvida para permitir que a Swift analise avaliações de clientes de forma automatizada, visual, interativa e inteligente.

O sistema permitirá que o usuário faça upload de uma base de avaliações em formato **CSV ou Excel**, contendo dados de lojas, notas, comentários opcionais, classificação dos clientes e informações relacionadas à gerência. Após o envio do arquivo, a aplicação realizará o processamento dos dados em memória, calculará métricas descritivas importantes, identificará possíveis outliers e aplicará modelos de Inteligência Artificial para interpretar os comentários dos clientes.

A solução terá como foco principal a análise de **NPS**, comparação entre tipos de gerência e interpretação automatizada dos comentários por meio de técnicas de **Processamento de Linguagem Natural**, também conhecido como NLP.

3. Objetivo Geral

Desenvolver uma aplicação web capaz de processar bases de avaliações da Swift, gerar indicadores gerenciais e aplicar modelos de Inteligência Artificial para apoiar a tomada de decisão sobre satisfação do cliente, desempenho das lojas e qualidade da operação.

4. Objetivos Específicos

O NeuroAnalytics terá como objetivos específicos:

1. Permitir o upload de arquivos CSV ou Excel contendo avaliações de lojas.
2. Realizar limpeza, padronização e validação dos dados enviados.

3. Calcular o NPS geral, por loja, por gerente e por tipo de gerência.
 4. Classificar os clientes em Promotores, Neutros e Detratores.
 5. Comparar o desempenho entre lojas com gerência Regular e Tocadora/Germinare.
 6. Identificar outliers que possam distorcer as análises e os modelos.
 7. Aplicar uma rede neural para análise de sentimento dos comentários.
 8. Aplicar um segundo modelo de NLP para categorizar os comentários por área de negócio.
 9. Exibir dashboards interativos com gráficos, tabelas e indicadores.
 10. Permitir que o usuário escolha se deseja ou não salvar uma análise no histórico.
 11. Armazenar apenas análises autorizadas pelo usuário.
 12. Disponibilizar uma tela de histórico com resumos das análises salvas.
 13. Realizar deploy da aplicação em ambiente web utilizando Docker, Render e GitHub Actions.
-

5. Justificativa do Projeto

Atualmente, a Swift utiliza o NPS como um indicador de satisfação do cliente e qualidade da operação das lojas. No entanto, a análise manual das avaliações pode ser limitada, principalmente quando há grande volume de dados, comentários textuais e diferenças entre tipos de gerência.

O NeuroAnalytics busca resolver esse problema ao centralizar a análise em uma aplicação web capaz de transformar uma base de avaliações em informações úteis para gestão.

A solução permitirá observar:

- quais lojas possuem melhor ou pior desempenho;
- se gerentes Regulares e Tocadores/Germinare apresentam diferenças relevantes;
- quais categorias aparecem com maior frequência nos comentários;
- quais sentimentos predominam nas avaliações;
- quais lojas possuem poucas avaliações e podem precisar impulsionar maior volume de respostas;
- quais dados podem ser outliers e distorcer os indicadores.

Dessa forma, o projeto contribui para uma visão mais clara da experiência do cliente e apoia decisões estratégicas relacionadas à operação das lojas.

6. Público-Alvo

O sistema será utilizado por pessoas envolvidas na análise de desempenho das lojas da Swift, como:

- gestores de loja;
 - analistas de dados;
 - equipes de negócio;
 - responsáveis por acompanhamento de NPS;
 - equipes de tecnologia e inovação;
 - lideranças interessadas em comparar performance entre lojas e tipos de gerência.
-

7. Escopo do Projeto

7.1. O que o sistema deve fazer

O sistema deverá:

- receber arquivos CSV ou Excel;
- processar os dados em memória;
- validar colunas obrigatórias;
- limpar dados inconsistentes;
- tratar comentários ausentes;
- identificar outliers;
- calcular métricas de NPS;
- classificar clientes por nota;
- separar resultados por loja;
- separar resultados por tipo de gerência;
- aplicar modelos de NLP nos comentários;
- classificar sentimentos em Positivo, Neutro ou Negativo;
- categorizar comentários em áreas de negócio;
- retornar os dados processados para o front-end em formato JSON;
- exibir dashboards interativos;
- permitir salvar ou descartar a análise;
- manter histórico apenas das análises salvas.

7.2. O que o sistema não deve fazer nesta versão

Nesta primeira versão, o sistema não terá como foco:

- autenticação avançada com múltiplos perfis de usuário;
- edição manual dos dados dentro da plataforma;
- treinamento pesado de modelos diretamente no ambiente de produção;
- uso de modelos grandes de linguagem hospedados localmente;
- armazenamento automático de todas as bases enviadas;
- integração direta com sistemas internos da Swift;
- envio automático de relatórios por e-mail;
- aplicativo mobile.

Esses recursos poderão ser considerados em versões futuras.

8. Dados de Entrada

A aplicação deverá aceitar arquivos nos formatos:

- .csv
- .xlsx
- .xls

A base deverá conter informações relacionadas às avaliações de clientes das lojas Swift.

8.1. Campos esperados na base

Os principais campos esperados são:

Campo	Descrição
Mes Ano	Mês e ano em que a avaliação foi realizada
Classificação	Nota atribuída pelo cliente
Data Avaliação	Data da avaliação em formato DD/MM/AA ou equivalente
CentroNv2	Identificação da loja Swift
qtd_clientes	Quantidade de clientes associada ao agrupamento
Comentário	Comentário opcional do cliente
Flag	Tipo de gerência, como REGULAR ou TOCADORA

8.2. Observações sobre a base

A base analisada contempla o período de janeiro de 2025 até janeiro de 2026. Existem 228 lojas, sendo 58 associadas a gerentes Germinare/Tocadora e 158 associadas a gerentes Regulares, conforme informações fornecidas no levantamento inicial.

O sistema deve considerar que pode existir apenas uma avaliação por mês em determinados agrupamentos e que algumas lojas podem ter volume de avaliações muito menor do que outras. Isso é importante, pois uma loja com poucas avaliações pode apresentar um NPS mais instável e suscetível a distorções.

9. Regras de Negócio

9.1. Classificação dos clientes

Os clientes deverão ser classificados de acordo com a nota atribuída:

Faixa de nota	Classificação
9 a 10	Promotor
7 a 8	Neutro
1 a 6	Detrator

Essa classificação será utilizada para cálculo do NPS e construção dos dashboards.

9.2. Cálculo do NPS

O NPS será calculado a partir da diferença entre a quantidade de promotores e detratores, dividida pelo total de avaliações válidas, retornando um valor percentual.

Fórmula recomendada para o sistema:

$$\text{NPS} = ((\text{Promotores} - \text{Detratores}) / \text{Total de Avaliações Válidas}) * 100$$

Caso a base esteja agrupada pelo campo `qtd_clientes`, o sistema deverá considerar a soma de `qtd_clientes` como total de avaliações válidas.

Exemplo:

```
Promotores = 800
Detratores = 200
Total de Avaliações = 1200

NPS = ((800 - 200) / 1200) * 100
NPS = 50%
```

9.3. Atenção sobre o denominador do NPS

Durante a especificação, deve-se validar com o cliente se o denominador oficial será:

- total de avaliações válidas;
- soma do campo `qtd_clientes`;
- média de avaliações por agrupamento.

Para fins de implementação técnica, a recomendação é utilizar o total de avaliações válidas ou a soma de `qtd_clientes`, pois essa abordagem segue o cálculo tradicional de NPS e evita distorções.

9.4. Separação por tipo de gerência

O sistema deverá separar os resultados conforme a coluna `Flag`.

As principais categorias serão:

Flag	Interpretação
REGULAR	Gerente regular, não Germinare

Flag	Interpretação
TOCADORA	Gerente Tocadora/Business/Germinare

A análise deverá permitir comparar:

- NPS médio por tipo de gerência;
- quantidade de avaliações por tipo de gerência;
- distribuição de promotores, neutros e detratores;
- principais categorias de comentário;
- sentimento predominante em cada grupo;
- lojas com melhor e pior desempenho em cada tipo de gerência.

9.5. Regra de salvamento no histórico

O sistema não deverá salvar automaticamente todas as análises processadas.

A API só deverá inserir dados no PostgreSQL quando receber do front-end o seguinte parâmetro:

```
{
  "save_analysis": true
}
```

Caso `save_analysis` seja `false` ou esteja ausente, a análise será processada, retornada ao front-end e descartada após o fim da sessão.

Essa regra tem como objetivo:

- reduzir consumo de banco de dados;
- evitar armazenamento desnecessário;
- economizar recursos na hospedagem gratuita;
- manter o histórico apenas com análises realmente relevantes.

9.6. Volatilidade dos dados

O processamento da base deverá ocorrer em memória RAM.

Fluxo esperado:

1. O usuário faz upload do arquivo.
2. O back-end recebe o arquivo.
3. O Python processa os dados em memória.
4. Os modelos de IA realizam inferência nos comentários.

5. O back-end retorna um JSON para o front-end.
 6. O front-end exibe os dashboards.
 7. Os dados só serão salvos se o usuário clicar em “Salvar Análise no Histórico”.
 8. Caso contrário, os dados são descartados.
-

10. Tratamento de Dados

10.1. Limpeza da base

A etapa de limpeza deverá incluir:

- remoção de linhas completamente vazias;
- padronização dos nomes das colunas;
- conversão de datas;
- conversão de notas para formato numérico;
- tratamento de comentários vazios;
- padronização da coluna `Flag`;
- remoção de duplicidades quando aplicável;
- validação do campo de loja;
- validação do campo de quantidade de clientes.

10.2. Tratamento de comentários ausentes

Como o comentário é opcional, o sistema deverá lidar com valores nulos.

Regras sugeridas:

- se o comentário estiver vazio, ele não será enviado aos modelos de NLP;
- avaliações sem comentário ainda entram no cálculo de NPS;
- dashboards textuais devem considerar apenas avaliações com comentário válido;
- o sistema pode exibir a porcentagem de avaliações com e sem comentário.

10.3. Identificação de outliers

A identificação de outliers é uma etapa crítica, pois valores extremos podem distorcer tanto as métricas descritivas quanto o treinamento ou inferência dos modelos.

O sistema deverá identificar outliers relacionados a:

- lojas com quantidade muito baixa de avaliações;
- lojas com quantidade muito alta de avaliações;
- notas extremamente concentradas em apenas uma classificação;
- meses com comportamento muito diferente do padrão;
- gerências com desempenho muito discrepante;
- comentários duplicados ou repetitivos.

Métodos sugeridos:

- intervalo interquartil;
- z-score;
- análise por percentis;
- comparação entre volume de avaliações e NPS;
- identificação de lojas com baixa representatividade estatística.

10.4. Análise de representatividade

O sistema deverá indicar quando uma loja possui poucas avaliações e, portanto, pode ter um NPS menos confiável.

Exemplo de alerta:

A Loja X possui baixo volume de avaliações no período analisado. Seu NPS pode estar mais sensível a avaliações extremas.

Esse recurso é importante porque uma loja pode parecer muito boa ou muito ruim apenas por possuir uma quantidade reduzida de avaliações.

11. Inteligência Artificial e NLP

O projeto utilizará técnicas de Processamento de Linguagem Natural para analisar os comentários opcionais deixados pelos clientes.

Serão desenvolvidos dois modelos principais:

1. Modelo de análise de sentimento.
2. Modelo de categorização por área de negócio.

11.1. Rede Neural 1 — Análise de Sentimentos

A primeira rede neural terá como objetivo classificar o tom geral dos comentários.

Classes previstas:

Classe	Descrição
Positivo	Comentário favorável à experiência
Neutro	Comentário sem emoção clara ou apenas informativo
Negativo	Comentário com crítica, reclamação ou insatisfação

Exemplos:

Comentário	Sentimento esperado
"Atendimento excelente e loja limpa."	Positivo
"Não encontrei o produto que queria."	Negativo
"Fui à loja durante a tarde."	Neutro

11.2. Rede Neural 2 — Categorização de Comentários

A segunda rede neural ou modelo de NLP deverá categorizar os comentários de acordo com a área de negócio mencionada.

Categorias iniciais:

Categoria	Descrição
Produto	Qualidade, variedade, preço ou características dos produtos
Limpeza	Higiene, organização e limpeza da loja
Atendimento	Experiência com funcionários, cordialidade e suporte
Abastecimento	Disponibilidade de produtos e reposição
Cultura de Produtos	Conhecimento, apresentação ou valorização dos produtos da Swift

Também poderá ser considerada uma categoria adicional:

Categoria	Descrição
Outros	Comentários que não se encaixam claramente nas categorias anteriores

11.3. Estratégia de modelagem

Como o ambiente gratuito de deploy pode ter limitações de memória e processamento, a modelagem deve priorizar modelos leves.

Opções recomendadas:

- modelos Mini ou Tiny de NLP;
- embeddings leves;
- modelos clássicos com TF-IDF como baseline;
- redes neurais simples para classificação;
- modelos quantizados;
- uso de batch pequeno na inferência;
- carregamento controlado dos modelos em memória.

11.4. Competição de Frameworks

Será realizada uma prova de conceito comparando PyTorch e TensorFlow.

O mesmo problema deverá ser treinado nos dois frameworks, utilizando a mesma base, mesmas divisões de treino/teste e mesmas métricas.

Critérios de comparação:

Critério	Descrição
Acurácia	Percentual de classificações corretas
F1-score	Equilíbrio entre precisão e revocação
Tempo de inferência	Tempo médio para classificar comentários
Uso de memória	Consumo de RAM durante execução
Facilidade de desenvolvimento	Simplicidade de implementação
Facilidade de deploy	Compatibilidade com Render/Docker
Tamanho do modelo	Peso final do arquivo do modelo

O framework escolhido será aquele que oferecer melhor equilíbrio entre desempenho, leveza e facilidade de manutenção.

12. Stack Tecnológico

12.1. Front-end

Tecnologia: React.js

Responsabilidades:

- tela de upload;
- dashboard interativo;
- gráficos;
- tabelas;
- botão de salvamento;
- tela de histórico;
- comunicação com a API;
- tratamento visual de erros;
- feedback de carregamento para o usuário.

Bibliotecas sugeridas:

- React;
- Vite;
- Axios;
- Recharts;
- React Dropzone;
- Tailwind CSS;
- TanStack Table.

12.2. Back-end

Tecnologia: FastAPI com Python

Responsabilidades:

- receber arquivos;
- validar formato;
- processar dados;
- calcular métricas;
- executar inferência nos modelos;
- retornar JSON ao front-end;
- salvar análises no banco quando solicitado;
- expor endpoints REST;
- lidar com erros de processamento.

Bibliotecas sugeridas:

- FastAPI;
- Uvicorn;
- Pandas;
- NumPy;
- Scikit-learn;
- PyTorch;
- TensorFlow;
- Pydantic;
- SQLAlchemy;
- Psycopg;
- Python Multipart;
- OpenPyXL.

12.3. Inteligência Artificial

Tecnologias: PyTorch e TensorFlow

Uso previsto:

- treinamento comparativo inicial;
 - análise de sentimentos;
 - categorização de comentários;
 - testes de performance;
 - escolha do framework final.
-

12.4. Banco de Dados

Tecnologia: PostgreSQL

Uso previsto:

- armazenar análises salvas;
 - armazenar metadados do arquivo;
 - armazenar métricas agregadas;
 - armazenar resultados por loja;
 - armazenar histórico de análises.
-

12.5. Infraestrutura

Tecnologias: Docker e Render

Uso previsto:

- containerização do front-end;
 - containerização do back-end;
 - deploy da API;
 - deploy do front-end;
 - conexão com banco PostgreSQL;
 - padronização do ambiente local e produção.
-

12.6. DevOps

Tecnologia: GitHub Actions

Uso previsto:

- rodar testes automatizados;
- validar build do React;
- validar endpoints do FastAPI;
- automatizar deploy;

- reduzir erros manuais.

13. Arquitetura Geral da Solução

A arquitetura do sistema será composta por:

1. Cliente acessando a aplicação React pelo navegador.
2. Front-end enviando arquivo para a API FastAPI.
3. Back-end processando o arquivo em memória.
4. Módulos de cálculo descritivo gerando indicadores.
5. Modelos de IA classificando comentários.
6. API retornando JSON estruturado para o front-end.
7. Front-end exibindo dashboards.
8. Usuário decidindo se deseja salvar a análise.
9. API salvando no PostgreSQL apenas se `save_analysis = true`.

13.1. Fluxo principal

```
Usuário
  ↓
Upload CSV/Excel no React
  ↓
FastAPI recebe o arquivo
  ↓
Pandas limpa e processa os dados
  ↓
Motor de métricas calcula NPS e estatísticas
  ↓
Modelos de NLP analisam comentários
  ↓
FastAPI retorna JSON
  ↓
React renderiza dashboard
  ↓
Usuário decide salvar
  ↓
PostgreSQL armazena histórico apenas se autorizado
```

14. Especificação do Back-end

14.1. Endpoints principais

POST `/api/upload`

Responsável por receber o arquivo e processar a análise.

Entrada:

- arquivo CSV ou Excel;
- parâmetro opcional `save_analysis`.

Saída:

- JSON com métricas calculadas;
- resultados por loja;
- resultados por tipo de gerência;
- sentimentos dos comentários;
- categorias dos comentários;
- alertas de outliers.

Exemplo de payload:

```
{
  "save_analysis": false
}
```

Exemplo de resposta:

```
{
  "summary": {
    "total_reviews": 12000,
    "nps geral": 48.5,
    "promoters": 7200,
    "neutral": 2600,
    "detractors": 2200
  },
  "management_comparison": [
    {
      "flag": "REGULAR",
      "nps": 52.1,
      "total_reviews": 8000
    },
    {
```

```
    "flag": "TOCADORA",
    "nps": 45.0,
    "total_reviews": 4000
  },
  ],
  "outliers": [
    {
      "store": "Loja X",
      "reason": "Baixo volume de avaliações"
    }
  ]
}
```

GET /api/history

Responsável por listar análises salvas.

Saída esperada:

```
[
  {
    "id": 1,
    "file_name": "avaliacoes_jan_2026.xlsx",
    "created_at": "2026-06-01T15:00:00",
    "nps": 48.5,
    "total_reviews": 12000
  }
]
```

GET /api/history/{id}

Responsável por buscar detalhes de uma análise salva.

DELETE /api/history/{id}

Responsável por apagar uma análise salva do histórico, caso esse recurso seja implementado.

GET /api/health

Endpoint simples para verificar se a API está ativa.

Resposta esperada:

```
{  
  "status": "ok"  
}
```

15. Especificação do Front-end

15.1. Tela inicial

A tela inicial deverá conter:

- nome do projeto;
- descrição breve da solução;
- área de upload com drag and drop;
- botão para selecionar arquivo;
- indicação dos formatos aceitos;
- mensagem de erro para arquivo inválido;
- botão para iniciar análise.

15.2. Tela de carregamento

Durante o processamento, o sistema deverá exibir:

- indicador de loading;
- mensagem informando que a base está sendo processada;
- aviso de que arquivos grandes podem demorar mais;
- bloqueio temporário de múltiplos envios simultâneos.

15.3. Dashboard de resultados

O dashboard deverá conter:

Indicadores principais

- NPS geral;
- total de avaliações;
- quantidade de promotores;
- quantidade de neutros;
- quantidade de detratores;
- percentual de comentários preenchidos;
- quantidade de lojas analisadas;
- quantidade de outliers encontrados.

Gráficos

- gráfico de barras com NPS por loja;
- gráfico de pizza ou rosca com distribuição Promotores/Neutros/Detratores;
- gráfico comparativo entre REGULAR e TOCADORA;
- ranking de melhores lojas;
- ranking de piores lojas;
- gráfico de categorias dos comentários;
- gráfico de sentimentos dos comentários.

Tabelas

- tabela por loja;
- tabela por tipo de gerência;
- tabela por gerente, se houver identificação disponível;
- tabela com outliers;
- tabela de comentários classificados pela IA.

Visualizações de NLP

- nuvem de palavras;
- distribuição por categoria;
- distribuição por sentimento;
- exemplos de comentários negativos;
- exemplos de comentários positivos;
- comentários mais recorrentes por área.

15.4. Botão de salvamento

Ao final do dashboard, deverá existir um botão visível:

Salvar Análise no Histórico

Ao clicar, o front-end deverá enviar uma requisição para a API com:

```
{  
  "save_analysis": true  
}
```

A aplicação deverá exibir confirmação visual após o salvamento.

Exemplo:

Análise salva com sucesso no histórico.

15.5. Tela de histórico

A tela de histórico deverá conter:

- lista de análises salvas;
- data de criação;
- nome do arquivo;
- NPS geral;
- total de avaliações;
- botão para visualizar detalhes;
- possível botão para excluir histórico.

16. Banco de Dados

16.1. Entidades principais

Tabela `analyses`

Armazena informações gerais da análise salva.

Campos sugeridos:

Campo	Tipo	Descrição
id	UUID ou SERIAL	Identificador da análise
file_name	VARCHAR	Nome do arquivo enviado
created_at	TIMESTAMP	Data de criação
total_reviews	INTEGER	Total de avaliações
general_nps	FLOAT	NPS geral
promoters	INTEGER	Total de promotores
neutral	INTEGER	Total de neutros
detractors	INTEGER	Total de detratores
saved	BOOLEAN	Indica se a análise foi salva

Tabela `store_results`

Armazena resultados agregados por loja.

Campos sugeridos:

Campo	Tipo	Descrição
id	UUID ou SERIAL	Identificador
analysis_id	FK	Referência à análise
store_name	VARCHAR	Nome/código da loja
flag	VARCHAR	Tipo de gerência
total_reviews	INTEGER	Total de avaliações
nps	FLOAT	NPS da loja
promoters	INTEGER	Promotores
neutral	INTEGER	Neutros
detractors	INTEGER	Detratores
is_outlier	BOOLEAN	Indica se é outlier

Tabela `comment_results`

Armazena comentários processados, caso seja necessário manter os detalhes.

Campos sugeridos:

Campo	Tipo	Descrição
id	UUID ou SERIAL	Identificador
analysis_id	FK	Referência à análise
store_name	VARCHAR	Loja
comment_text	TEXT	Comentário
sentiment	VARCHAR	Sentimento previsto
category	VARCHAR	Categoria prevista
confidence	FLOAT	Confiança do modelo

Tabela management_summary

Armazena resumos por tipo de gerência.

Campos sugeridos:

Campo	Tipo	Descrição
id	UUID ou SERIAL	Identificador
analysis_id	FK	Referência à análise
flag	VARCHAR	REGULAR ou TOCADORA
total_reviews	INTEGER	Total de avaliações
nps	FLOAT	NPS do grupo
promoters	INTEGER	Promotores
neutral	INTEGER	Neutros
detractors	INTEGER	Detratores

17. Requisitos Funcionais

Código	Requisito
RF01	O sistema deve permitir upload de arquivos CSV e Excel
RF02	O sistema deve validar se o arquivo possui as colunas obrigatórias
RF03	O sistema deve limpar e padronizar os dados recebidos
RF04	O sistema deve calcular o NPS geral
RF05	O sistema deve calcular o NPS por loja
RF06	O sistema deve calcular o NPS por tipo de gerência
RF07	O sistema deve classificar clientes como Promotores, Neutros ou Detratores
RF08	O sistema deve identificar outliers
RF09	O sistema deve aplicar análise de sentimento nos comentários
RF10	O sistema deve categorizar comentários por área
RF11	O sistema deve retornar os resultados em JSON
RF12	O sistema deve exibir dashboards interativos
RF13	O sistema deve permitir salvar uma análise no histórico

Código	Requisito
RF14	O sistema deve listar análises salvas
RF15	O sistema deve processar dados em memória quando a análise não for salva

18. Requisitos Não Funcionais

Código	Requisito
RNF01	A aplicação deve ser responsiva
RNF02	A API deve ser assíncrona
RNF03	O sistema deve evitar consumo excessivo de memória
RNF04	Os modelos de IA devem ser otimizados para deploy leve
RNF05	O sistema deve usar Docker
RNF06	O sistema deve possuir pipeline de CI/CD
RNF07	O front-end deve apresentar mensagens claras de erro
RNF08	A API deve validar entradas antes do processamento
RNF09	O sistema deve ser hospedado em ambiente web
RNF10	O banco deve armazenar apenas análises salvas
RNF11	O sistema deve ter endpoints documentados pelo Swagger do FastAPI
RNF12	O tempo de resposta deve ser aceitável para arquivos de tamanho moderado

19. DevOps e Deploy

19.1. Docker

O projeto deverá possuir:

- `Dockerfile` para o front-end React;
- `Dockerfile` para o back-end FastAPI;
- `docker-compose.yml` para ambiente local;
- configuração de variáveis de ambiente;
- separação entre ambiente de desenvolvimento e produção.

19.2. Docker Compose

Serviços previstos:

```
services:
  frontend:
    build: ./frontend
    ports:
      - "3000:3000"

  backend:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=${DATABASE_URL}

  database:
    image: postgres
    environment:
      - POSTGRES_USER=neuro
      - POSTGRES_PASSWORD=neuro
      - POSTGRES_DB=neuroanalytics
```

19.3. GitHub Actions

A pipeline deverá executar:

- instalação de dependências;
- testes do back-end;
- testes de endpoints;
- validação da build do front-end;
- build das imagens Docker;
- deploy automático na Render.

19.4. Render

A aplicação deverá ser hospedada na Render, com serviços separados:

- serviço web para o front-end;
- serviço web para a API FastAPI;
- banco PostgreSQL;
- variáveis de ambiente configuradas no painel da plataforma.

Como o plano gratuito pode ter limitações de memória, a aplicação deverá evitar modelos muito grandes e processamentos pesados em produção.

20. Segurança e Validações

O sistema deverá implementar validações básicas para evitar falhas e uso incorreto.

Validações necessárias:

- verificar extensão do arquivo;
- limitar tamanho máximo do arquivo;
- validar colunas obrigatórias;
- impedir upload de arquivos vazios;
- tratar comentários nulos;
- tratar notas fora do intervalo esperado;
- proteger credenciais com variáveis de ambiente;
- evitar exposição de dados sensíveis;
- retornar mensagens de erro controladas.

Exemplo de erro:

```
{
  "error": "Arquivo inválido",
  "detail": "A coluna 'Classificação' não foi encontrada."
}
```

21. Critérios de Aceite

O projeto será considerado funcional quando:

1. O usuário conseguir enviar uma planilha pelo front-end.
 2. O back-end conseguir processar o arquivo sem salvar automaticamente.
 3. O sistema calcular corretamente o NPS geral.
 4. O sistema calcular NPS por loja.
 5. O sistema separar análises entre REGULAR e TOCADORA.
 6. O sistema identificar possíveis outliers.
 7. O sistema classificar comentários por sentimento.
 8. O sistema categorizar comentários por área de negócio.
 9. O dashboard exibir gráficos e tabelas compreensíveis.
 10. O usuário conseguir salvar uma análise no histórico.
 11. O histórico listar análises salvas.
 12. O banco armazenar somente análises autorizadas.
 13. O projeto rodar localmente com Docker.
 14. A pipeline do GitHub Actions validar build e testes.
 15. O deploy estar acessível em ambiente web.
-

22. Riscos do Projeto

Risco	Impacto	Mitigação
Modelo de IA muito pesado	API pode cair em produção	Usar modelos Mini/Tiny ou quantizados
Base com muitos dados inconsistentes	Métricas incorretas	Criar etapa robusta de validação
Poucos comentários preenchidos	NLP pouco representativo	Exibir taxa de comentários válidos
Baixa quantidade de avaliações por loja	NPS distorcido	Criar alerta de baixa representatividade
Limite de memória da hospedagem	Falha na inferência	Otimizar modelos e processamento
Diferença entre regra oficial e regra implementada do NPS	Resultado divergente	Validar fórmula final com o cliente
Comentários ambíguos	Classificação incorreta	Usar classe "Outros" e confiança do modelo

23. Melhorias Futuras

Em versões futuras, o NeuroAnalytics poderá incluir:

- login de usuários;
- permissões por perfil;
- exportação de relatórios em PDF;
- envio automático de relatórios por e-mail;
- integração com sistemas internos da Swift;
- comparação temporal entre meses;
- previsão de NPS futuro;
- recomendação automática de ações corretivas;
- uso de modelos de linguagem mais avançados via API externa;
- painel administrativo;
- upload de múltiplos arquivos;
- auditoria de análises salvas.

24. Conclusão

O NeuroAnalytics será uma solução completa para análise de avaliações de lojas da Swift, combinando dashboards, métricas de NPS, comparação entre tipos de gerência e Inteligência Artificial aplicada aos comentários dos clientes.

A aplicação permitirá transformar uma base de avaliações em insights claros, ajudando a identificar pontos fortes, problemas recorrentes, lojas com desempenho crítico, diferenças entre gerências e áreas que mais impactam a satisfação do cliente.

Com React no front-end, FastAPI no back-end, PostgreSQL para histórico, modelos de NLP para análise textual e Docker com GitHub Actions para deploy, o projeto terá uma arquitetura moderna, escalável e adequada para uma primeira versão funcional em ambiente web.

O principal diferencial do sistema será unir análise quantitativa e qualitativa: além de calcular indicadores como NPS, o NeuroAnalytics também interpretará os comentários dos clientes, permitindo uma visão mais profunda sobre a experiência nas lojas Swift.